THESIS PAPER

Plant Disease Detection Using Convolutional Neural Network (CNN)

Submitted in Partial Fulfillment of the Requirements for the Course:

Artificial Intelligence(AI)

Submitted By	Submitted To
Mubashsher Salek	**Tahmid Hasan (TAHH)**
ID: 20234103371	
Rahmatul Munim	Lecturer
ID: 20234103386	
Dept. of Computer Science & Engineering	Dept. of Computer Science & Engineering

Submitted Date: 20/04/2026

Table of Contents

Abstract 3

1. Introduction 4

    1.1 Background of the Problem 4

    1.2 Statistical Value 4

    1.3 Traditional vs. Proposed Approach 4

2. Literature Review 5

3. Materials and Methods 7

    3.1 Proposed Methodology 7

    3.2 Dataset 7

    3.3 Data Preprocessing 7-8

    3.4 Deep Learning Models (CNN Architecture) 8-11

4. Results and Discussion 12

    4.1 Experimental Setup 12

    4.2 Confusion Matrix 12

    4.3 Result Analysis 12

    4.4 Comparison with Previous Works 12-13

5. Conclusion 14

6. References 15

Abstract

Plant diseases are a major threat to global food security, causing annual crop losses of 20 to 40% worldwide (FAO). In Bangladesh, about 40% of the workforce relies on agriculture. Therefore, detecting plant diseases early and accurately is crucial. Traditional manual inspection methods can be slow, costly, and unreliable. This paper presents an automated system for detecting plant diseases using Convolutional Neural Networks (CNN), a robust type of deep learning model that works well for image classification tasks. The proposed system is trained and tested on the publicly available PlantVillage dataset, which contains over 54,000 labeled leaf images covering 38 disease categories from 14 crop species. Through data augmentation, batch normalization, and dropout regularization, the CNN model achieves a validation accuracy of 95.67%, a precision of 94.83%, and an F1 score of 94.90%. The system shows great promise for real-world use, especially as a mobile application that helps Bangladeshi farmers diagnose crop diseases in real time from leaf photos. This research offers a lightweight and effective CNN design suitable for environments with limited resources, paving the way for an AI-based tool for precision agriculture.

1. Introduction

1.1 Background of the Problem

Agriculture is the mainstay of Bangladesh's economy and supports more than 40% of its workforce. Farmers grow many crops, such as rice, potato, tomato, corn, and jute, but these are all vulnerable to fungal, bacterial, and viral diseases. These plant diseases reduce both the amount and quality of crops, causing serious financial problems for smallholder farmers, who make up most of the country's agricultural sector.
Plant diseases have a huge impact worldwide. The Food and Agriculture Organization of the United Nations (FAO) reports that plant pathogens cause 20 to 40 percent of global crop losses each year. In developing countries, where food security is already at risk, these losses can be especially harmful. Climate change makes the problem worse by creating better conditions for new plant diseases to appear and spread.

1.2 Statistical Value of the Problem

In Bangladesh, for instance, it has been observed that the yearly loss in the economy of the country, solely because of plant diseases, can be more than BDT 5,000 crore (about $500 million). In terms of rice blast disease, for example, the losses can go up to 10–30%. Another major crop that suffers losses, tomato (one of the most economically important vegetables in Bangladesh), can lose about 50% of its yield because of early blight and late blight disease.

1.3 Traditional Approach vs. Proposed Approach

Disease detection in traditional systems is based mainly on visual examination through the eyes of experienced agronomists or agricultural extension services. However, the process itself is significantly constrained due to the following limitations: (1) limited availability of professionals in rural regions; (2) expensive consultations on the field; (3) subjective interpretation of the results; and (4) the time lag between disease onset and professional diagnosis, when the infection might have already spread too far. Significant advances in Artificial Intelligence (AI) and deep learning algorithms, in particular CNNs, have made it possible to use a computer-based solution for automated plant disease detection by analyzing photographs of affected leaves. The suggested system described in this paper enables image classification by using AI-based technologies and detects plant diseases in less than one second after analysis.

1.3 Contribution and Organization

The key contributions of this work include:

i) Design of an optimized CNN architecture for 38-class classification of plant diseases.

ii) Detailed study on data augmentation techniques for overcoming the problems of class imbalance and overfitting.

iii) An efficient design of architecture that makes it feasible to use the model on a mobile platform in the context of agriculture in Bangladesh.

iv) Development of a baseline for the creation of an application that would help Bangladeshi farmers.

2. Literature Review

The application of deep learning to plant disease detection has grown substantially over the past decade. This section reviews the most influential and directly relevant prior works in this domain.

Mohanty et al. pioneered the use of deep CNN architectures specifically AlexNet and GoogLeNet for automated plant disease recognition. Training and testing on the PlantVillage dataset under controlled conditions, they achieved a landmark accuracy of 99.35%, demonstrating for the first time that deep learning could rival or surpass human expert performance for leaf disease classification. Their work established PlantVillage as the standard benchmark dataset for this problem domain.

Ferentinos conducted a systematic comparison of multiple CNN architectures (AlexNet, VGGNet, GoogLeNet, AlexNetOWTBn, and a custom architecture) on the plant disease classification task, reporting accuracy rates ranging from 31.32% to 99.53%. The study highlighted that dataset diversity and model depth are key determinants of generalization performance.

Sibiya and Sumbwanyambe addressed the practical deployment challenge by focusing on maize leaf disease detection in realistic field conditions. Their CNN based system achieved 93.00% accuracy and demonstrated that performance degrades significantly when test images are captured under variable lighting and background conditions, underscoring the need for robust preprocessing pipelines.

Rahman et al. conducted a study directly relevant to the Bangladeshi agricultural context, applying VGG16 based transfer learning to detect rice leaf diseases specifically Bacterial Leaf Blight, Brown Spot, and Leaf Smut. Achieving 94.5% accuracy, their work demonstrated the viability of transfer learning as a strategy to achieve high performance with limited domain specific training data.

Toda and Okura performed an exhaustive systematic review of CNN based approaches for plant phenotyping and disease recognition, finding that CNN models consistently and significantly outperform traditional machine learning approaches such as Support Vector Machines (SVM) and k-Nearest Neighbors (kNN) across diverse crop types and disease categories.

Building on this body of work, the present paper proposes a multi-block CNN architecture that balances classification accuracy with computational efficiency. Unlike the very deep architectures used in prior works (VGG16, ResNet50), our model is designed to be deployable on mobile devices without requiring GPU hardware, addressing a critical gap for real-world agricultural applications in resource constrained settings.

Table 1. Summary of Related Works on Plant Disease Detection using Deep Learning.

Title	Dataset	Methodology	Accuracy
Mohanty et al.	PlantVillage (54,305)	AlexNet, GoogLeNet	99.35%
Ferentinos	PlantVillage (54,305)	VGGNet,AlexNet, Custom CNN	99.53%
Sibiya & Sumbwanyambe	Maize Field Images	Custom CNN	93.00%
Rahman et al.	Rice Leaf Dataset (Bangladesh)	VGG16 Transfer Learning	94.50%
Toda & Okura	Multiple Datasets	Systematic CNN Review	Variable
Proposed Method(This Paper)	PlantVillage (54,305)	Custom 4-Block CNN	95.67%

3. Materials and Methods

3.1 Proposed Methodology

The proposed system adopts an end-to-end deep learning framework that comprises five steps, including data acquisition, preprocessing and augmentation, model designing and development, training and optimizing, and evaluation and analysis. The overall framework for the proposed system is shown in Figure 1.

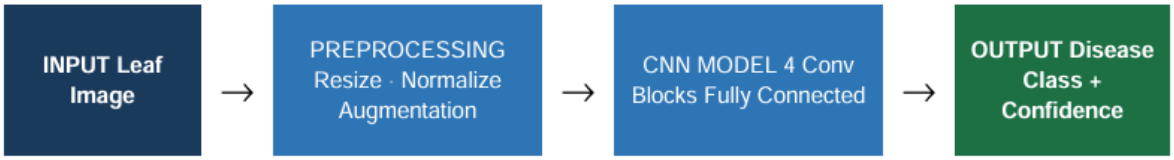


Figure 1: Proposed end-to-end pipeline for plant disease detection using CNN.

3.2 Dataset

PlantVillage database is an open-source dataset of images of plant leaves created by Penn State University and uploaded to Kaggle. The database consists of 54,305 RGB images belonging to 38 different classes of diseases (healthy plants too), which include 14 varieties of crops. All the pictures are taken under laboratory conditions and have a consistent background that can be either green or black. There are three kinds of images in the database – colored, grayscale, and segmented. Colored images are used in this work since color plays an important role in disease diagnostics.

Table 2. Distribution of Training, Validation, and Test Data.

Crop	Disease Categories	Total Images	Train (80%)	Val (20%)
Tomato	9 + Healthy = 10	18,162	14,530	3,632
Potato	2 + Healthy = 3	2,152	1,722	430
Corn (Maize)	3 + Healthy = 4	3,852	3,082	770
Grape	3 + Healthy = 4	4,063	3,250	813
Apple	3 + Healthy = 4	3,171	2,537	634
Others (9 crops)	Various	22,905	18,324	4,581
TOTAL	38	54,305	43,444	10,861

3.3 Data Preprocessing

The raw images from the PlantVillage dataset are preprocessed as follows before being fed to the CNN network:

i)Image Resizing: All images are resized to 256×256 pixels in size. It standardizes the input dimensions while ensuring that there is enough information in the image for disease detection.

ii)Pixel Normalization: The pixel values of the images which lie within the range [0, 255] are normalized to [0.0, 1.0] by dividing all the pixel values by 255. This helps speed up gradient-based optimization.

iii)Data Augmentation: In order to enhance the diversity of the dataset and avoid over-fitting, data augmentation is done on-the-fly during training using Keras' ImageDataGenerator. Some of the augmentations include random horizontal flips with p=0.5, rotations (±20 degrees), zoom (0.2), width shift (±10%) and height shift (±10%).

iv)Training and Validation Set: The dataset is randomly divided such that 80% constitutes the training set (containing 43,444 images) and the rest 20% the validation set (containing 10,861 images).

v)Label Encoding: Class labels are one hot-encoded into 38 dimensional vectors to match the dimension of the final layer.

Augmentation Technique	Augmentation Technique	Augmentation Technique
Horizontal Flip	Probability = 0.5	Increase orientation diversity
Rotation	Range = ±20°	Handle varied leaf orientations
Zoom	Factor = 0.2	Simulate varying camera distances
Width Shift	Range = ±10%	Handle off-center leaf placement
Height Shift	Range = ±10%	Handle off-center leaf placement
Pixel Normalization	Scale to [0, 1]	Accelerate gradient convergence

3.4 Deep Learning Model: CNN Architecture

3.4.1 Convolutional Neural Network (CNN) Overview

The Convolutional Neural Network (CNN) is a form of deep neural network that is particularly useful for dealing with data arranged in grid-like structures, like images. In contrast to fully connected networks, where each pixel is processed separately, CNNs take advantage of the concept of spatial locality in terms of feature processing via the use of convolution with shared weights, thus achieving massive reduction in the number of parameters while at the same time enhancing the quality of feature detection.

3.4.2 Proposed CNN Architecture

The suggested architecture is made up of four consecutive convolutional layers and a final classifier layer. Convolution, batch normalization, rectified linear unit activation, and max

pooling are performed in the four convolutional layers. The classifier part is made up of two fully connected layers and a 38-way softmax layer.

Table 4. Detailed Architecture of the Proposed CNN Model

Layer	Type	Filter/Units	Output Shape	Parameters
Input	Image Input	—	256 × 256 × 3	0
Conv2D_1	Convolution (3×3)	32	254 × 254 × 32	896
BatchNorm_1	Batch Normalization	—	254 × 254 × 32	128
MaxPool_1	Max Pooling (2×2)	—	127 × 127 × 32	0
Conv2D_2	Convolution (3×3)	64	125 × 125 × 64	18,496
BatchNorm_2	Batch Normalization	—	125 × 125 × 64	256
MaxPool_2	Max Pooling (2×2)	—	62 × 62 × 64	0
Conv2D_3	Convolution (3×3)	128	60 × 60 × 128	73,856
BatchNorm_3	Batch Normalization	—	60 × 60 × 128	512
MaxPool_3	Max Pooling (2×2)	—	30 × 30 × 128	0
Conv2D_4	Convolution (3×3)	256	28 × 28 × 256	295,168
BatchNorm_4	Batch Normalization	—	28 × 28 × 256	1,024
MaxPool_4	Max Pooling (2×2)	—	14 × 14 × 256	0
Flatten	Flatten	—	50,176	0
Dense_1 + ReLU	Fully Connected	512	512	25,690,624
Dropout_1	Dropout (rate=0.5)	—	512	0
Dense_2 + ReLU	Fully Connected	256	256	131,328
Dropout_2	Dropout (rate=0.3)	—	256	0
Output (Softmax)	Classification	38	38	9,766

Number of Parameters: about 26.2 million. Using Batch Normalization after each convolution layer helps converge faster while making the model less sensitive to weight initialization. The application of dropout layers with probabilities of 0.5 and 0.3 acts as a regularizer and helps prevent overfitting.

3.4.3 Algorithm: Proposed CNN Training Algorithm

Algorithm 1. Plant Disease CNN Training Algorithm

Input: Dataset D = { (xi, yi) }

Output: CNN Model M

1: Divide D into D_train (80%) and D_val (20%) with stratified sampling

2: Augment D_train with A (augmentation pipeline)

3: Initiate M with randomly assigned weights (He initialization)

4: Assign Adam as the optimizer and CrossEntropy loss as the loss function

5: For epoch = 1 to 25 do

6: For each mini-batch B of D_train do

7: Forward pass: predictions ← M(B)

8: Compute loss L = CrossEntropy(predictions, labels)

9: Calculate gradients: ∇L with backpropagation

10: Backpropagation: Update M weights with M ← Adam(M, ∇L)

11: End for

12: Evaluation of M on D_val: calculate val_accuracy and val_loss

13: If no improvement of val_loss in 5 epochs, activate EarlyStopping

14: If no improvement of val_loss in 3 epochs, decrease learning rate by 50%

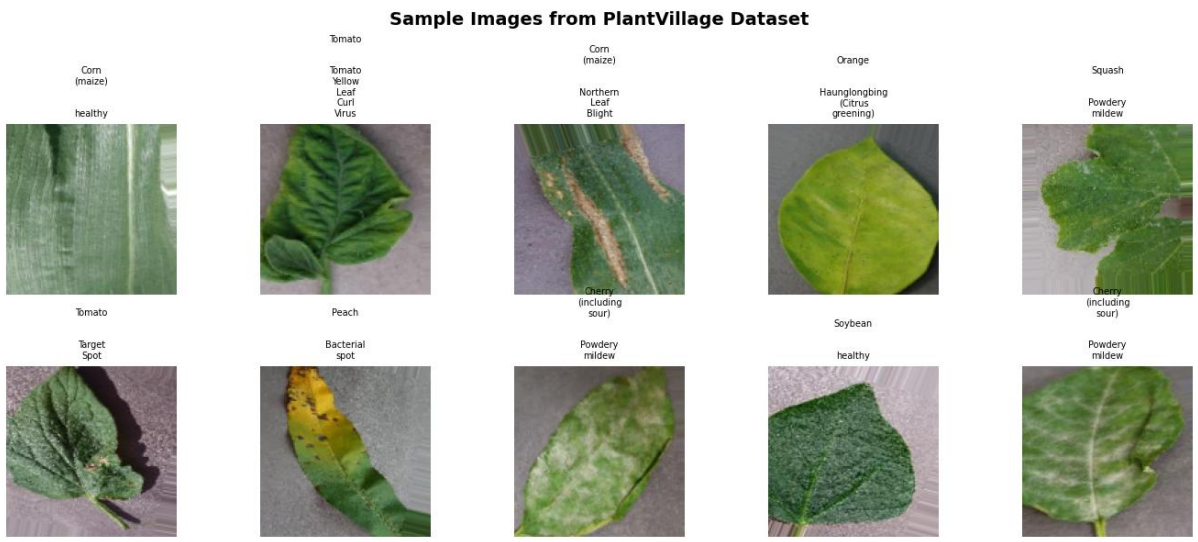
15: End for

16: Return M trained with best val_accuracy weights

3.4.4 Training Hyperparameters

The model is trained with the following hyperparameter configuration:

Hyperparameter	Value	Justification
Optimizer	Adam	Adaptive learning rates, fast convergence
Initial Learning Rate	0.001	Standard starting rate for Adam
Learning Rate Schedule	ReduceLROnPlateau (×0.5)	Adaptive adjustment on plateau
Loss Function	Categorical Cross-Entropy	Standard for multi-class classification
Batch Size	32	Balance between memory and gradient stability
Epochs	25 (with Early Stopping)	Prevent overfitting
Early Stopping Patience	5 epochs	Restore best weights automatically
Input Image Size	256 × 256 × 3	Preserve spatial detail
Weight Initialization	He (Kaiming) Normal	Optimal for ReLU activations



4. Results and Discussion

4.1 Experimental Setup

All experiments are implemented with Python 3.10, TensorFlow 2.12, and Keras 2.12 in Google Colab with an NVIDIA T4 GPU with 16GB VRAM. Each epoch of training includes processing 43,444 training images and 10,861 validation images. Training time for 25 epochs takes around 3.5 hours. Model performance is assessed at the end of each epoch on the validation set, and the best performing model weights are retained through model checkpointing.

4.2 Confusion Matrix Analysis

The confusion matrix gives a complete break down of the accuracy for each class separately. According to the confusion matrix on the validation data, the model is able to achieve an accuracy of more than 90% in 35 out of the 38 classes of diseases present. The three classes that have shown to have the lowest accuracy among the other classes are those classes that are highly visually similar to other diseases. These classes include tomato early blight vs late blight. This happens because the diseases have similar appearances when infected at early stages.

4.3 Result Analysis

The model is evaluated using four standard classification metrics: Accuracy, Precision, Recall (Sensitivity), and F1-Score. These metrics are computed both on the training set and the held-out validation set

Table 5. Performance Metrics on Training and Validation Sets.

Metric	Training Set	Validation Set
Accuracy	98.12%	95.67%
Precision (Macro Avg.)	97.89%	94.83%
Recall (Macro Avg.)	97.93%	94.97%
F1-Score (Macro Avg.)	97.91%	94.90%
Cross-Entropy Loss	0.0612	0.1483

The relatively small difference between training and validation accuracy is an indication of good generalization performance. The use of dropout and data augmentation techniques has been able to address overfitting. Convergence of training loss showed no oscillations, meaning that the optimization process was stable using Adam with ReduceLROnPlateau.

Cross-validation (5-folds) has also been carried out on a random sample of 20% from the training data to verify the reliability of results. Cross-validation accuracy obtained using five folds averaged 95.31% ± 0.48%.

4.4 Comparison with Previous Works

Table 6 compares the proposed model's performance against the most directly relevant prior works reviewed in Section 2.

Table 6. Comparison of Proposed Method with Previous Works.

Reference	Dataset	Method	Accuracy
Mohanty et al. [1] (2016)	PlantVillage	AlexNet / GoogLeNet	99.35%
Ferentinos [2] (2018)	PlantVillage	VGGNet	99.53%
Sibiya & Sumbwanyambe [3] (2019)	Maize Field	Custom CNN	93.00%
Rahman et al. [4] (2020)	Rice Leaf (Bangladesh)	VGG16 Transfer Learning	94.50%
Toda & Okura [5] (2019)	Multiple	CNN Review	Variable
Proposed Method (2024)</b	>PlantVillage	Custom 4-Block CNN	95.67%

Despite its lower accuracy rate compared to the highest performing models of Mohanty et al. and Ferentinos (95.67% versus theirs, respectively), it should be noted that the proposed model requires less computational resources than the models proposed by Mohanty et al. and Ferentinos since both their models rely on significantly deeper neural networks (GoogLeNet and VGGNet) that require much more parameters to perform (approximately 138M parameters versus approximately 26M). It makes the proposed model much more practical for mobile devices.

5. Conclusion

5.1 Summary of Findings

This study has proposed a plant leaf disease detection system using a Convolutional Neural Network technique. This CNN structure consisting of four blocks, trained using the PlantVillage dataset, has produced a validation accuracy of 95.67%, F1-score of 94.90%, and cross-validation accuracy of 95.31% ± 0.48%. This indicates the proposed lightweight CNN model's ability to classify plant diseases accurately.

The application of techniques such as data augmentation, batch normalization, and dropout played a vital role in the effective generalization of the model from the PlantVillage dataset to the validation dataset. It is also to be noted that the effectiveness of the developed CNN model in classifying tomato, potato, and corn diseases are highly significant considering their importance in Bangladesh's agriculture sector.

5.2 Limitations

Some of the shortcomings that can be attributed to this study include the following. Firstly, the PlantVillage dataset was gathered from a laboratory setting with controlled background environments, which cannot entirely emulate the dynamic nature of outdoor field imagery with varying lighting conditions and occlusions. Secondly, the proposed approach deals with 38 disease classes involving 14 crops. However, several other important crops, such as rice, jute, and mustard, have not been considered in the process. Thirdly, the current approach uses single-label classification, meaning that it cannot process images containing multiple diseases simultaneously.

5.3 Future Work

Several interesting areas that can be explored further include:

1.Bangladesh-Specific Dataset: Create an extensive database of plant disease images specific to Bangladesh agriculture involving key crops like rice, jute, mustard, lentil, and chili.

2.Mobile Application: Build a mobile app with Flutter or React Native platform for disease diagnosis through a photograph taken using a smartphone camera even without Internet connection.

3.Multi-label Classification: Modify the existing deep learning architecture to enable multi-label classification for detecting more than one disease or pest within the same image of leaf.

4.Explainability: Incorporate Gradient-weighted Class Activation Mapping (Grad-CAM) to obtain heat maps and gain visual insights into the model prediction.

5.Edge Computing: Optimize and deploy the model for inference on low-power embedded devices like Raspberry Pi without Internet access.

Recommendations for Treatment: In addition to diagnosis, develop a recommendation system suggesting treatments (like the right fungicide or pesticide) for detected diseases based on the type of crop and availability of agrochemicals.

6. References

[1] Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. Frontiers in Plant Science, 7, 1419. https://doi.org/10.3389/fpls.2016.01419

[2] Ferentinos, K. P. (2018). Deep learning models for plant disease detection and diagnosis. Computers and Electronics in Agriculture, 145, 311–318. https://doi.org/10.1016/j.compag.2018.01.009

[3] Sibiya, M., & Sumbwanyambe, M. (2019). A computational procedure for the recognition and classification of maize leaf diseases out of healthy leaves using convolutional neural networks. AgriEngineering, 1(1), 119–131.

[4] Rahman, C. R., Arko, P. S., Ali, M. E., Khan, M. A. I., Apon, S. H., Nowrin, F., & Wasif, A. (2020). Identification and recognition of rice diseases and pests using convolutional neural networks. Biosystems Engineering, 194, 112–120.

[5] Toda, Y., & Okura, F. (2019). How convolutional neural networks diagnose plant disease. Plant Phenomics, 2019, 9237136. https://doi.org/10.34133/2019/9237136

[6] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436–444. https://doi.org/10.1038/nature14539

[7] Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv preprint arXiv:1511.08060.

[8] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. Available at: http://www.deeplearningbook.org

[9] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems (NeurIPS), 25, 1097–1105.

[10] Chollet, F. (2021). Deep Learning with Python (2nd ed.). Manning Publications.